MIT School of Computing, MIT ADT University

Applied Edge Analytics

AIEC Mini Project

INDUSTRIAL IoT
EDGE MACHINE-LEARNING SYSTEM
On

Applied Edge Analytics

Submitted to

MIT ADT University, Pune

in partial fulfilment of the requirements for the Assignment of the course

Applied Edge Analytics

Submitted by

Tejaswa Gupta (ADT23SOCB1228)

Roll No: 36

Under the Guidance of

Prof. Minakshi Sonawane



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MIT SCHOOL OF COMPUTING, MIT ADT UNIVERSITY

RAJBAUG, LONI KALBHOR, PUNE – 412201

FINAL YEAR - SOC A.Y. 2026-27 SEMESTER-VII

Abstract

This project demonstrates an industrial predictive-maintenance machine-learning workflow. A synthetic sensor dataset is used to train a Random Forest classifier to distinguish normal operating conditions from failure-risk conditions. Azure Machine Learning runs the training job and stores the resulting model artifacts. The model is registered in Azure ML, downloaded, and loaded by a Python scoring module. A lightweight HTTP server exposes health and prediction endpoints. Both endpoints returned HTTP 200 during testing, and the supplied sample was classified as Normal with a reported failure probability of 0.0. The Docker package has been prepared, but container build and deployment to an IoT Edge/EFLOW device remain unverified.

1. Introduction

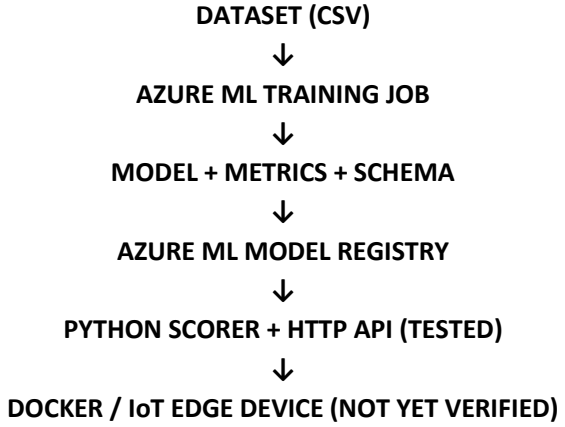
Industrial equipment generates measurements such as vibration, pressure, temperature, rotational speed, and accumulated runtime. Machine-learning models can use these measurements to estimate whether operating conditions resemble normal operation or a failure-risk state. This prototype covers model training, registration, and HTTP-based inference.

2. Objectives

- Train a binary classifier using industrial sensor features.
- Evaluate performance on a held-out test dataset.
- Register the trained model in Azure Machine Learning.
- Build a scoring interface returning failure class and probability.
- Expose health and prediction endpoints over HTTP.
- Prepare an edge-module package and document the remaining deployment work.

3. System Architecture

Verified flow: synthetic CSV dataset → Azure ML command job → model artifacts and metrics → Azure ML model registry → downloaded model → Python scorer → HTTP inference server. Intended next stage: sensor source → containerized inference on an edge host → monitoring/alerts. The final device connection remains pending.



4. Dataset and Input Schema

The dataset contains 2,400 synthetic records, with 1,920 training rows and 480 test rows reported by the training job. The five numeric features are vibration, pressure, temperature, rpm, and runtime_hours. The binary target is failure, where 0 means Normal and 1 means Failure Risk.

Feature	Type	Description
vibration	Numeric	Vibration measurement
pressure	Numeric	Operating pressure
temperature	Numeric	Temperature measurement
rpm	Numeric	Rotational speed
runtime_hours	Numeric	Accumulated runtime
failure	Binary target	0 = Normal; 1 = Failure Risk

5. Model Training and Evaluation

The training script uses a Random Forest classifier with a preprocessing pipeline. Azure ML job joyful_insect_v23v724bqp completed on compute target iot-cpu-cluster. The output directory contained model.joblib, metrics.json, and schema.json.

Metric	Value	Percentage
Accuracy	0.9833	98.33%
Precision	0.9701	97.01%
Recall	0.9155	91.55%
F1 score	0.9420	94.20%
ROC-AUC	0.9977	99.77%

Confusion matrix (actual rows, predicted columns):

	Predicted Normal	Predicted Failure Risk
Actual Normal	407	2
Actual Failure Risk	6	65

The model correctly classified 407 normal samples and 65 failure-risk samples. It produced 2 false positives and missed 6 failure-risk samples. Since the dataset is synthetic, these metrics are not proof of performance on real industrial machinery.

6. Azure ML Implementation

- Resource group: IndustrialIoTML-RG.
- Workspace: industrial-iot-ml-ws; reported location: polandcentral.
- Compute: iot-cpu-cluster; minimum nodes 0, maximum nodes 1.
- Training run: joyful_insect_v23v724bqp; status Completed.
- Registered model: machine-failure-classifier, version 1, type custom_model.
- Artifacts downloaded: model.joblib, metrics.json, schema.json.

7. Inference Implementation and Test

The scoring module loads the model with joblib and expects the five numeric features listed above. It returns failure, failure_label, and failure_probability. The HTTP server exposes GET /health and POST /predict on port 8081 in Cloud Shell.

Test input:

```
{"vibration":0.25,"pressure":100.0,"temperature":65.0,"rpm":1500,"runtime_hours":1200}
```

Observed prediction:

```
{"predictions":[{"failure":0,"failure_label":"Normal","failure_probability":0.0}]}
```

The health endpoint returned HTTP 200 with {"status":"healthy"}. The prediction endpoint also returned HTTP 200. The model initially failed when loaded with scikit-learn 1.9.1; using version 1.7.2, matching the training environment, resolved the compatibility issue.

8. Container Preparation and Limitations

A Dockerfile and requirements file were created for Python 3.12, pinning scikit-learn==1.7.2. The archive IndustrialIoTEdgeModule.tar.gz was created in Cloud Shell at approximately 387 KB. Cloud Shell had a Docker client but no accessible daemon. Docker Desktop on Windows was confirmed to be running, but the archive transfer did not succeed. Therefore, Docker image build, container tests, image push, and IoT Edge/EFLOW deployment are not claimed as completed.

9. Results and Discussion

- Azure ML training completed and produced model artifacts.
- The model was registered as machine-failure-classifier:1.
- Evaluation reported 98.33% accuracy, 97.01% precision, 91.55% recall, 94.20% F1 score, and 99.77% ROC-AUC.
- Python inference succeeded after aligning scikit-learn to version 1.7.2.
- HTTP health and prediction endpoints both returned HTTP 200.
- The prototype still requires validation on real sensor data and, if required, deployment to an IoT Edge device.

10. Remaining Work

1. Transfer the archive to a Docker-enabled machine.
2. Build the Docker image and test /health and /predict inside the container.
3. If required, deploy the module to the target IoT Edge/EFLOW device.
4. Connect a sensor simulator or real sensor source.
5. Add a dashboard/alert interface if required by project scope.
6. Validate performance on real or independently collected industrial data.

11. Screenshot Evidence

Screenshots supplied during implementation are listed below. Captions describe the evidence each image is intended to support.

Figure 1. Azure Machine Learning workspace overview.

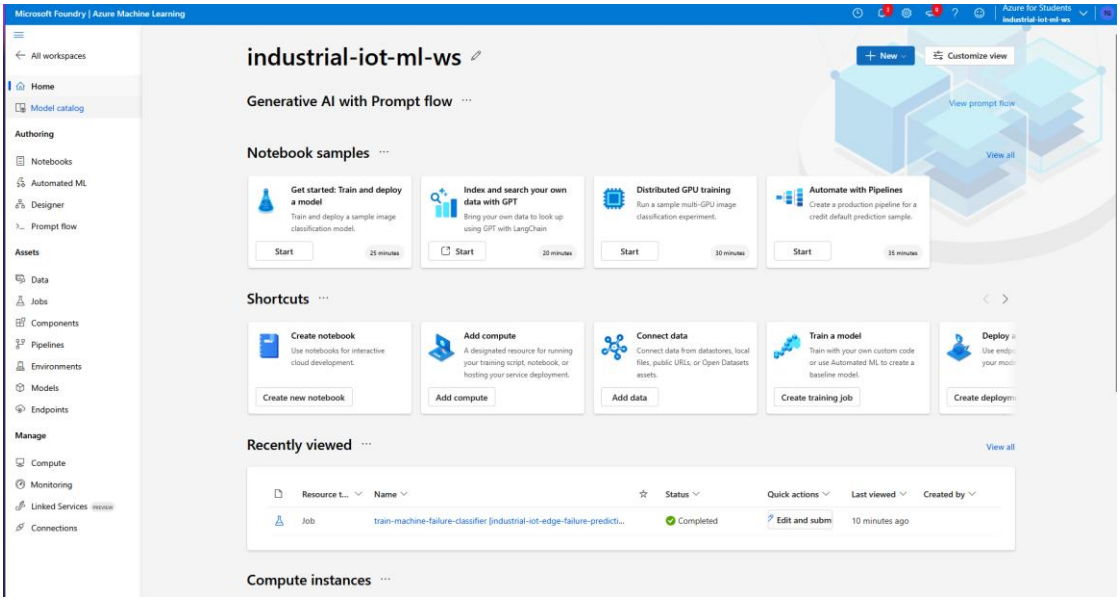


Figure 2. Completed Azure ML training job overview.

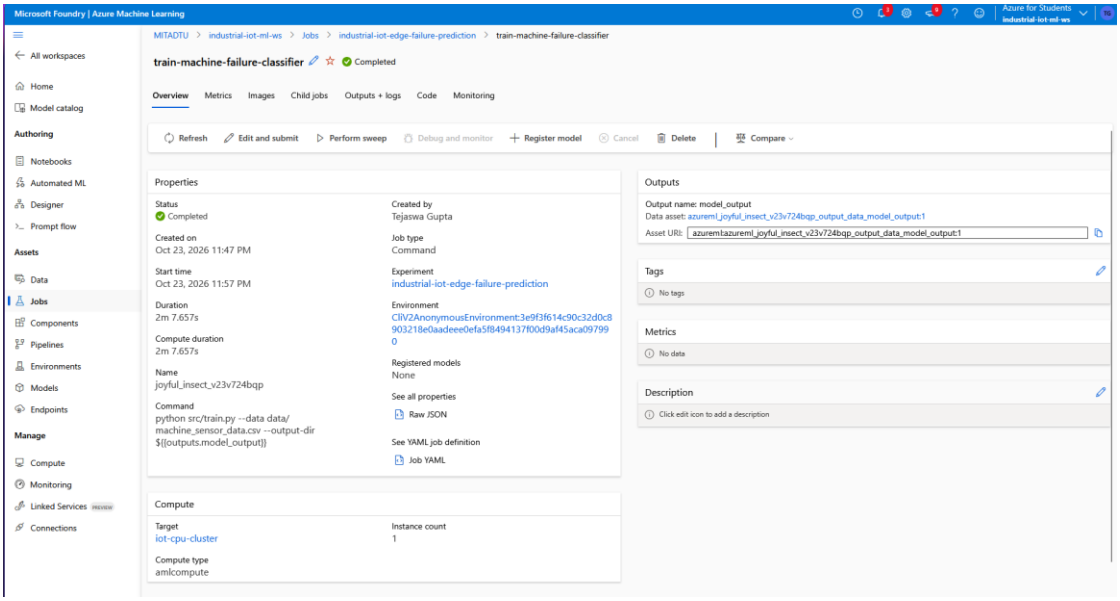


Figure 3. Model registration workflow.

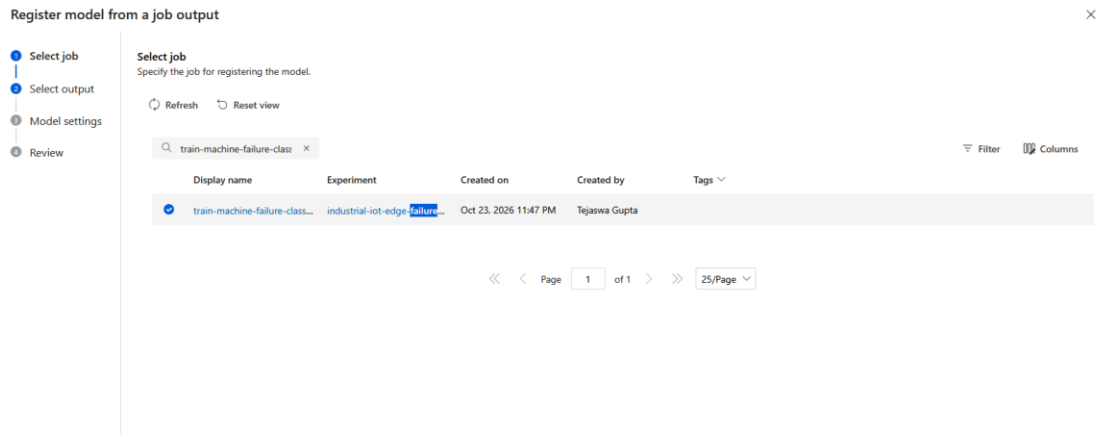


Figure 4. Training metrics and confusion matrix.

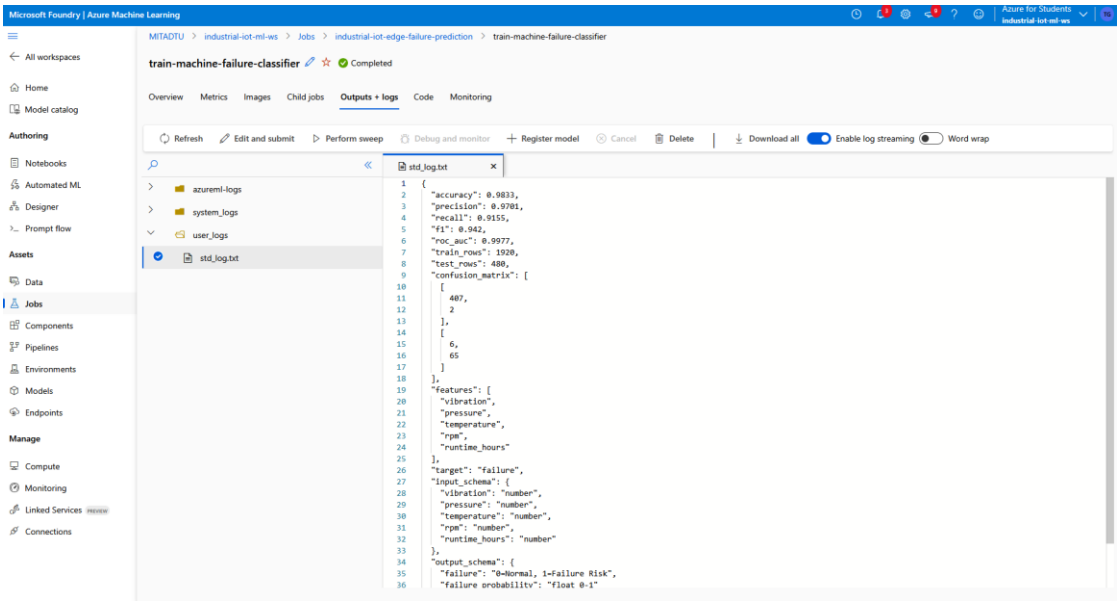


Figure 5. Successful inference after matching scikit-learn version.

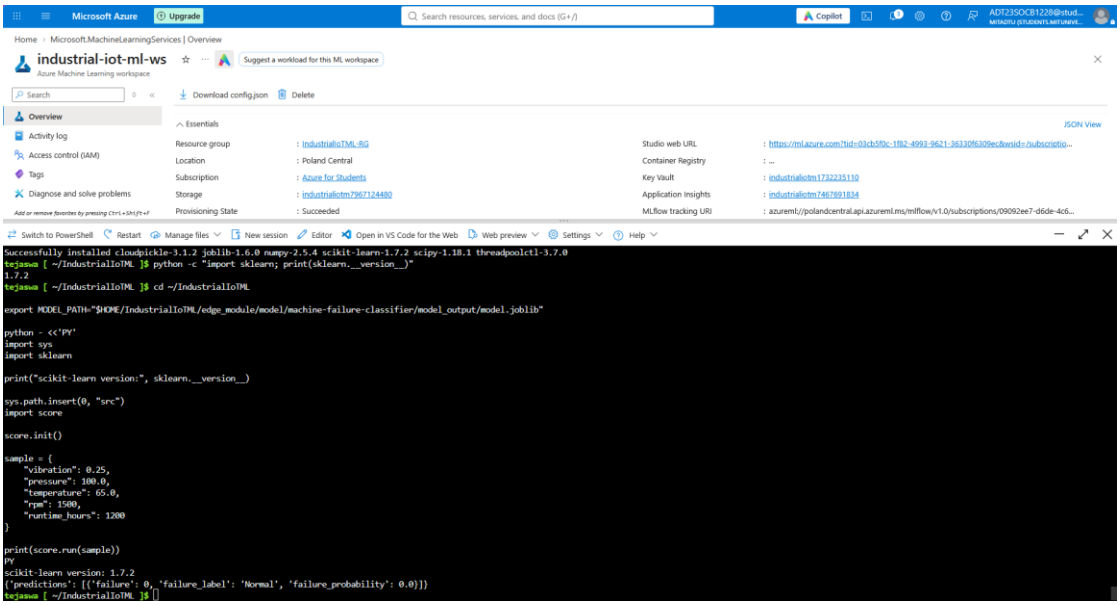


Figure 6. Registered model listing, version 1.

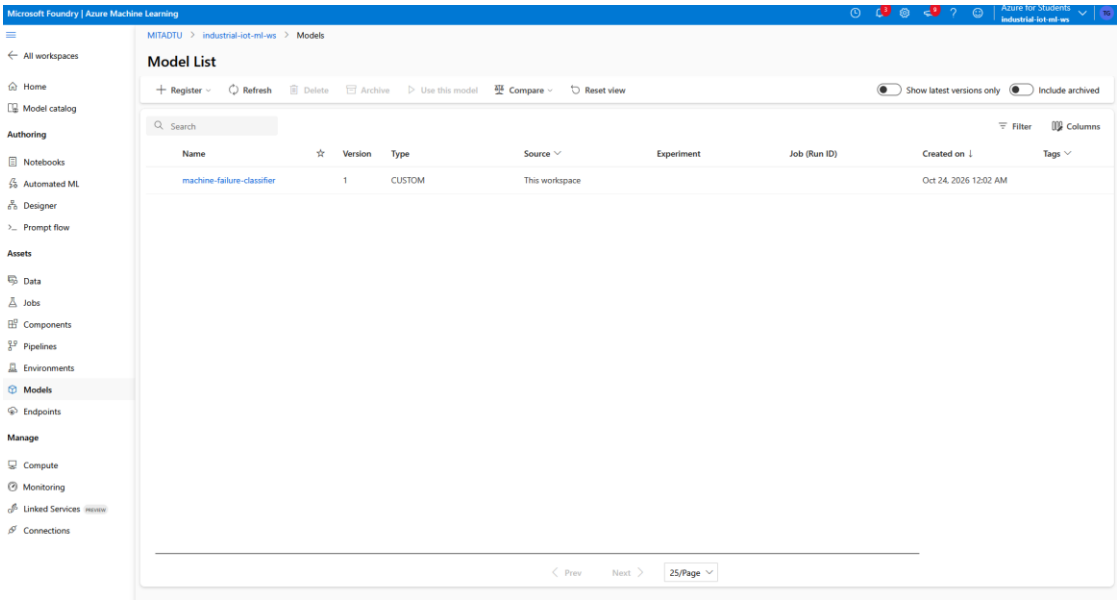


Figure 7. HTTP health endpoint returned 200 OK.

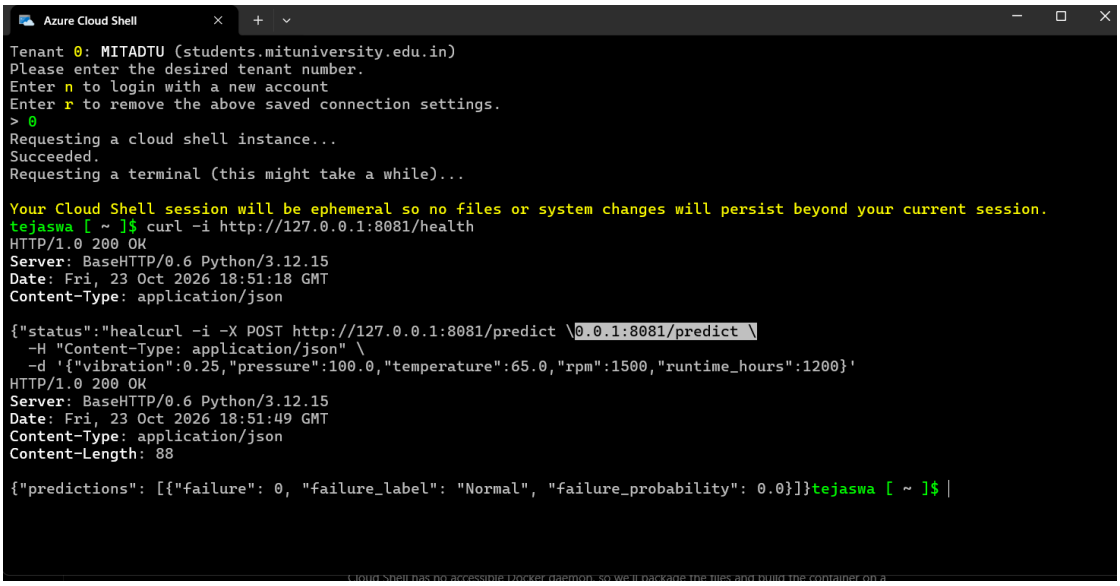


Figure 8. HTTP prediction endpoint returned 200 OK.

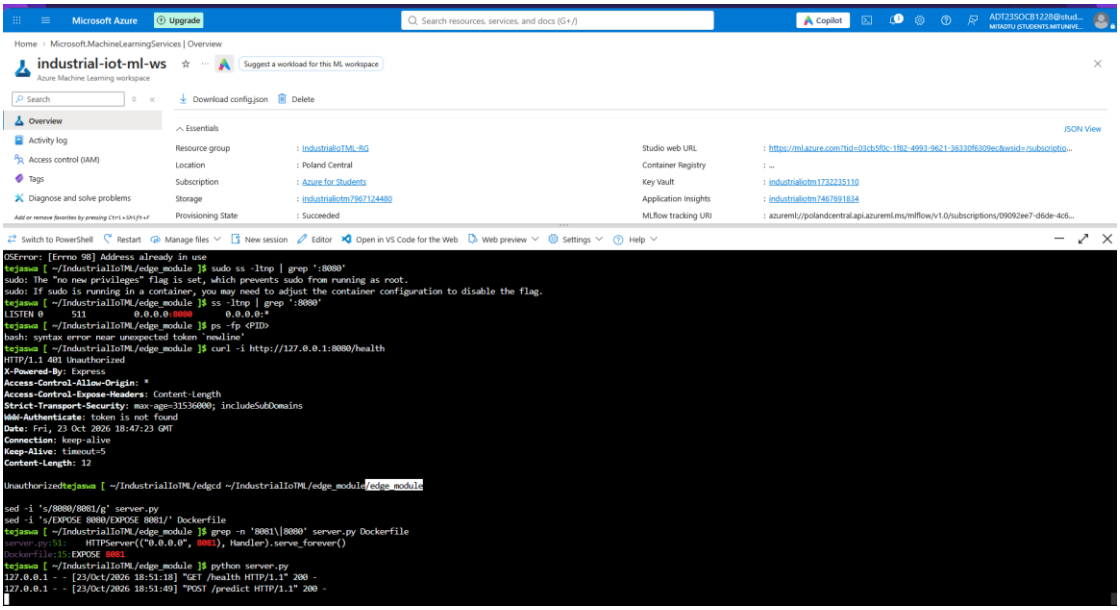
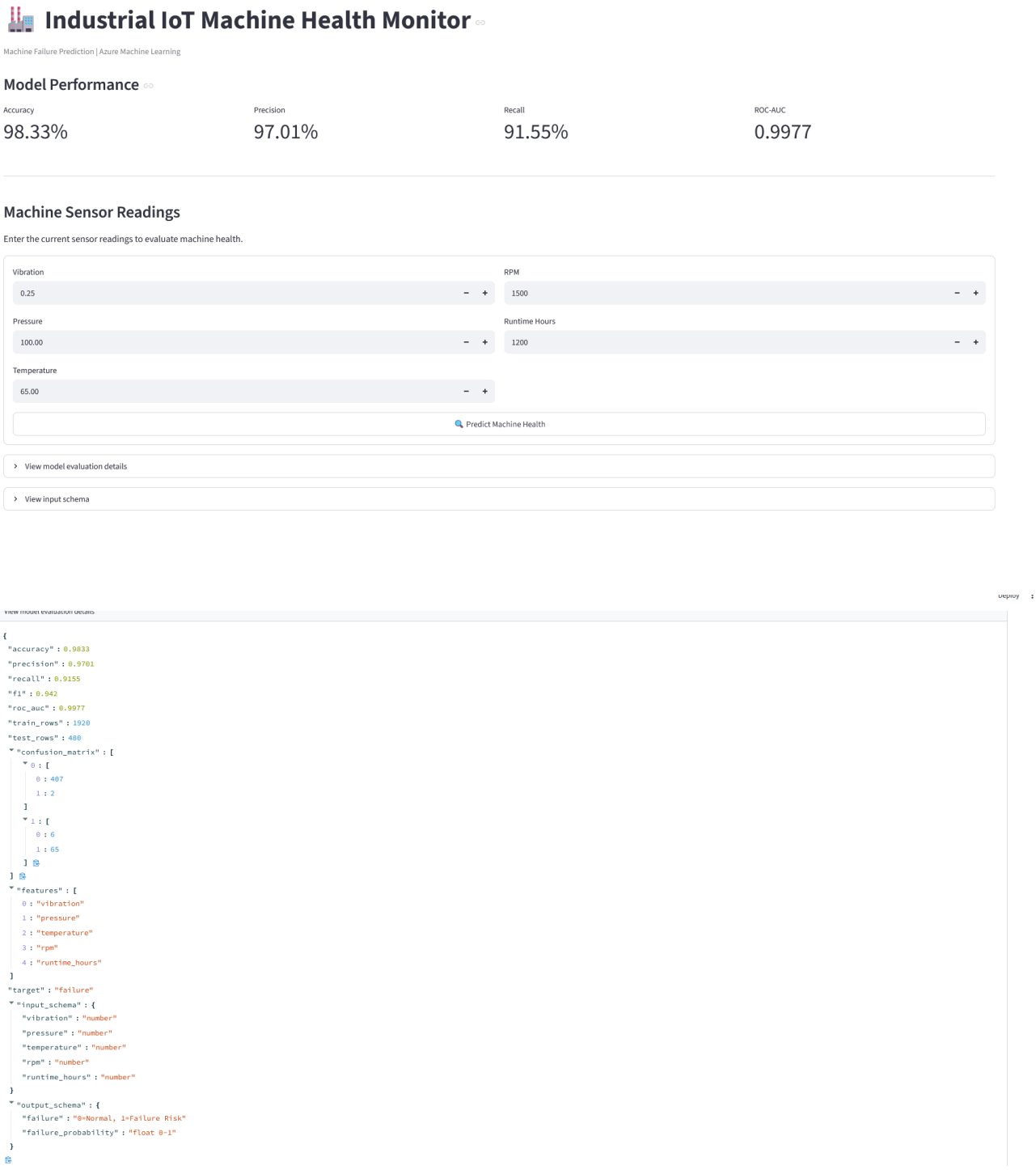
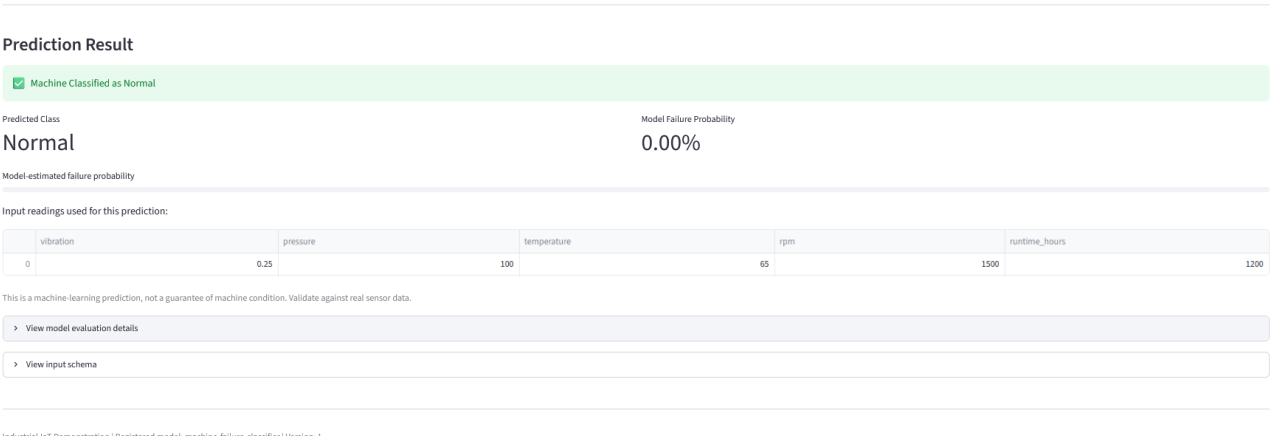


Figure 9. UI Streamlit





12. Conclusion

The project has established a functioning Azure ML training and model-registration workflow and a tested Python inference API. The model achieved 98.33% accuracy on the synthetic test set, with recall of 91.55%. HTTP health and prediction tests succeeded. The Docker package is prepared, but container build and IoT Edge deployment remain outstanding. These steps and real-sensor validation are the primary next milestones.

Appendix A — Key Identifiers

- Resource group: IndustrialIoTML-RG
- Workspace: industrial-iot-ml-ws
- Training run: joyful_insect_v23v724bqp
- Model: machine-failure-classifier:1
- Scoring file: src/score.py
- HTTP server: edge_module/server.py
- Configured port: 8501
- Model artifact: model.joblib